

Washing Machine Simulator

Lab 1

Things to define

- **Machine states**
- **Inputs**
- **Machine structure**
- **State transitions**

IDLE

- No washing cycle is running.
- User can select/change mode.
- Door can be open.
- User can press Start.

WAITING FOR DETERGENT

- Start has been requested.
- A valid wash mode has been selected.
- Door is closed.
- Detergent is missing.
- Machine waits until detergent is filled.
- Once detergent is filled, the machine automatically starts the wash cycle.

RUNNING

- Washing is in progress.
- Door is locked.
- Mode cannot change.
- Start has no effect.
- Timer counts down.

POWER FAILURE

- Washing is interrupted because power failed.
- The current machine state and remaining time are preserved.
- Door remains locked.
- Machine waits for power restoration.
- When power is restored, the machine resumes the unfinished wash cycle from the preserved remaining time.

COMPLETED

- Timer reaches zero.
- Washing is finished.
- Door is unlocked.
- Machine returns to **IDLE** and is ready for a new wash cycle.

ABORTED

- User pressed Abort during washing.
- Cycle stops.
- Door unlocks.
- Machine can return to IDLE for a new cycle.

User Inputs

- START
- ABORT
- SELECT_HEAVY
- SELECT_NORMAL
- SELECT_LIGHT
- DOOR_OPEN
- DOOR_CLOSE
- DETERGENT_FILLED
- POWER_OFF
- POWER_ON
- SHOW_STATUS

```
typedef struct
```

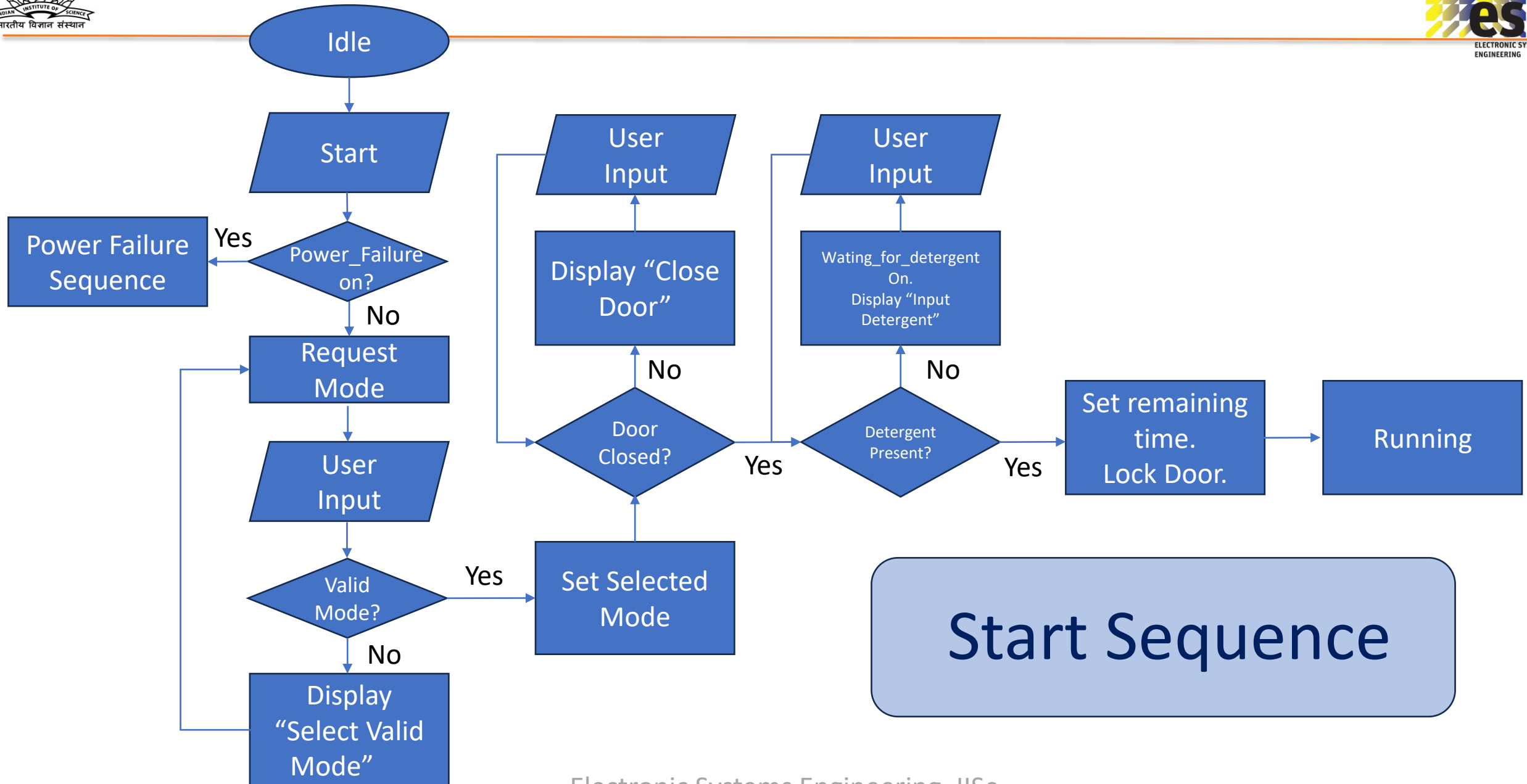
```
{  
    WashMode mode;  
    MachineState state;  
    DoorStatus door_status;  
    int remaining_time;  
    int detergent_present;  
    int start_requested;  
} WashingMachine;
```

```
typedef enum  
{  
    MODE_NONE,  
    MODE_HEAVY,  
    MODE_NORMAL,  
    MODE_LIGHT } WashMode;
```

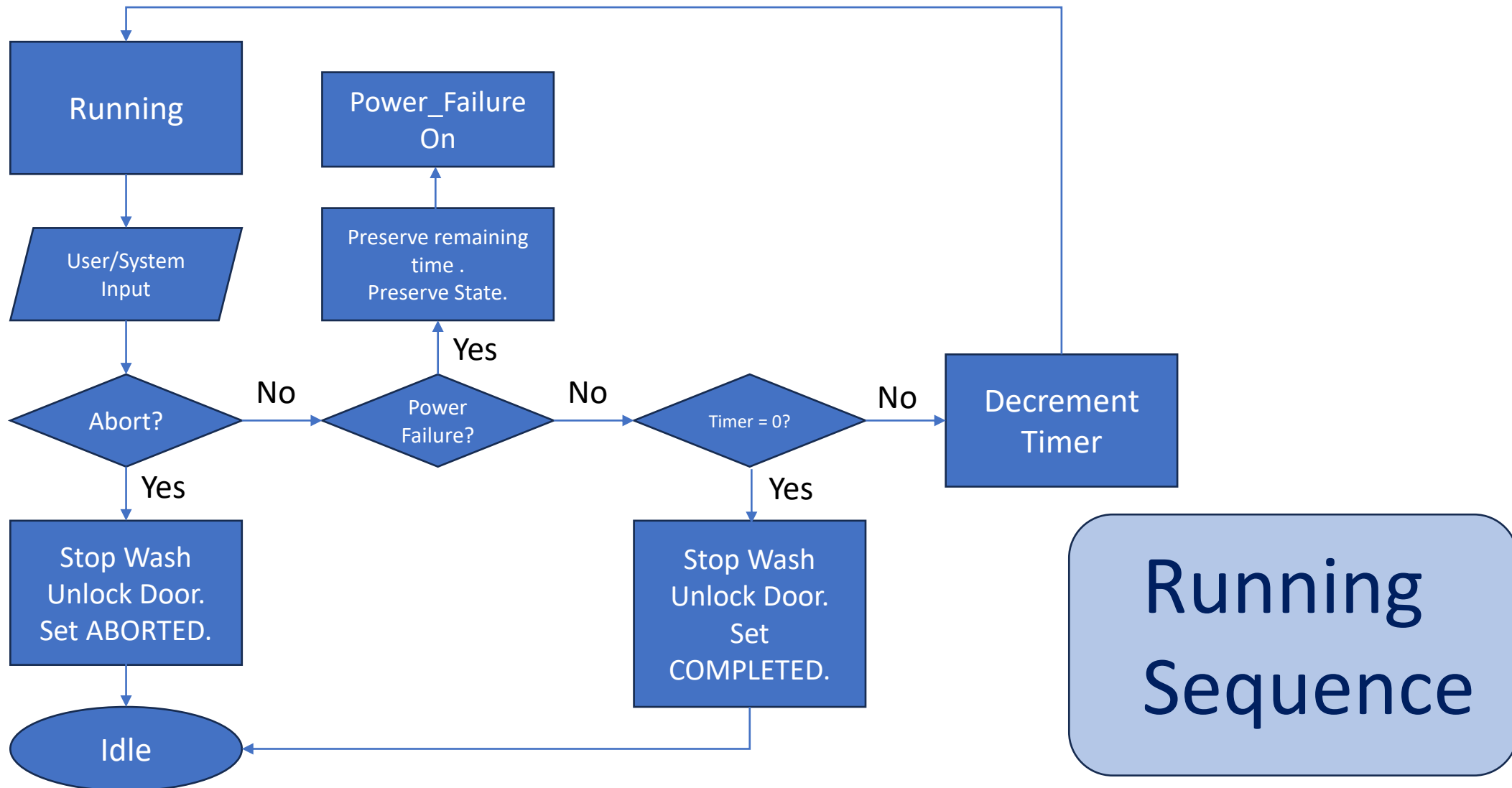
```
typedef enum  
{  
    DOOR_OPEN,  
    DOOR_CLOSED,  
    DOOR_LOCKED  
} DoorStatus;
```

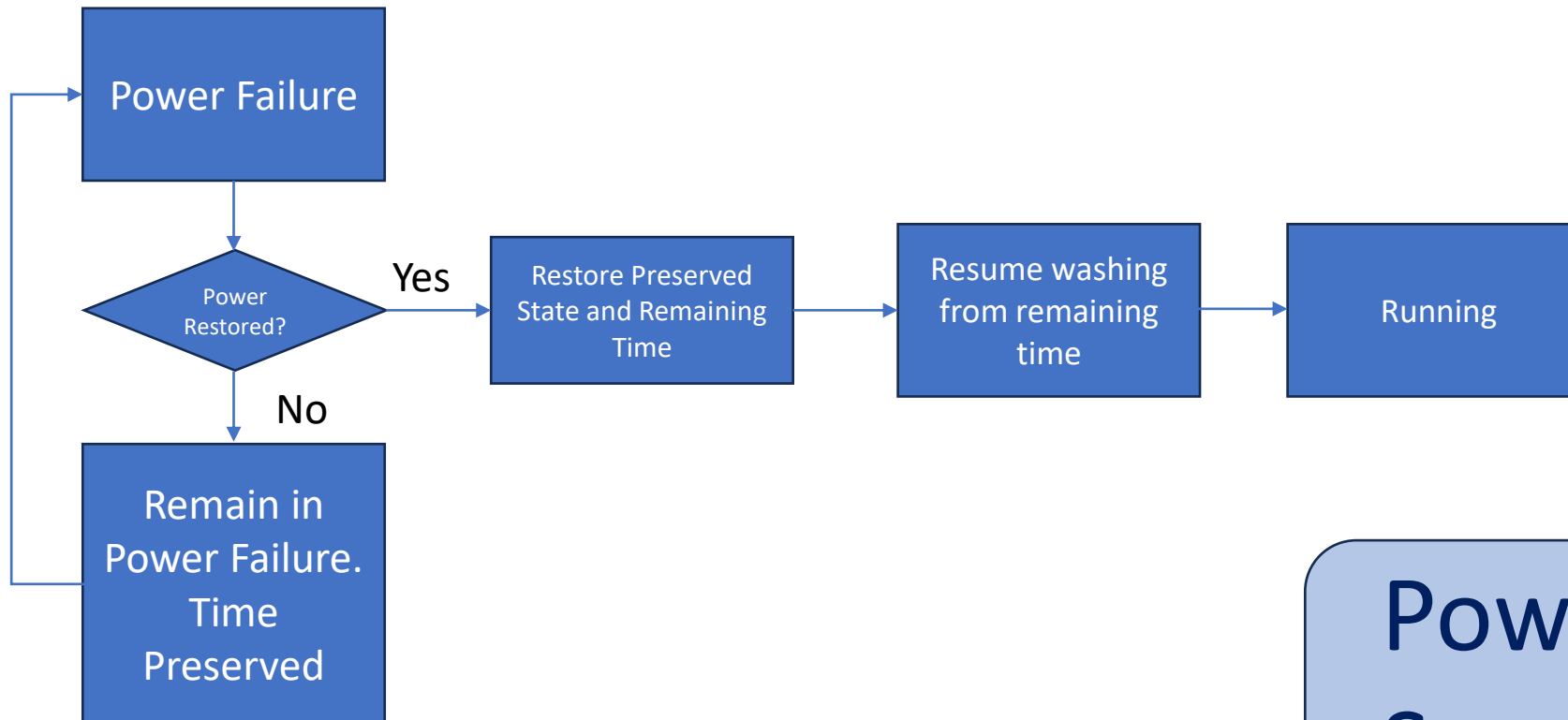
```
typedef enum  
{  
    IDLE,  
    WAITING_FOR_DETERGENT,  
    RUNNING,  
    POWER_FAILURE,  
    COMPLETED,  
    ABORTED  
} MachineState;
```

- Start Sequence
- Running Sequence
- Power Failure Sequence

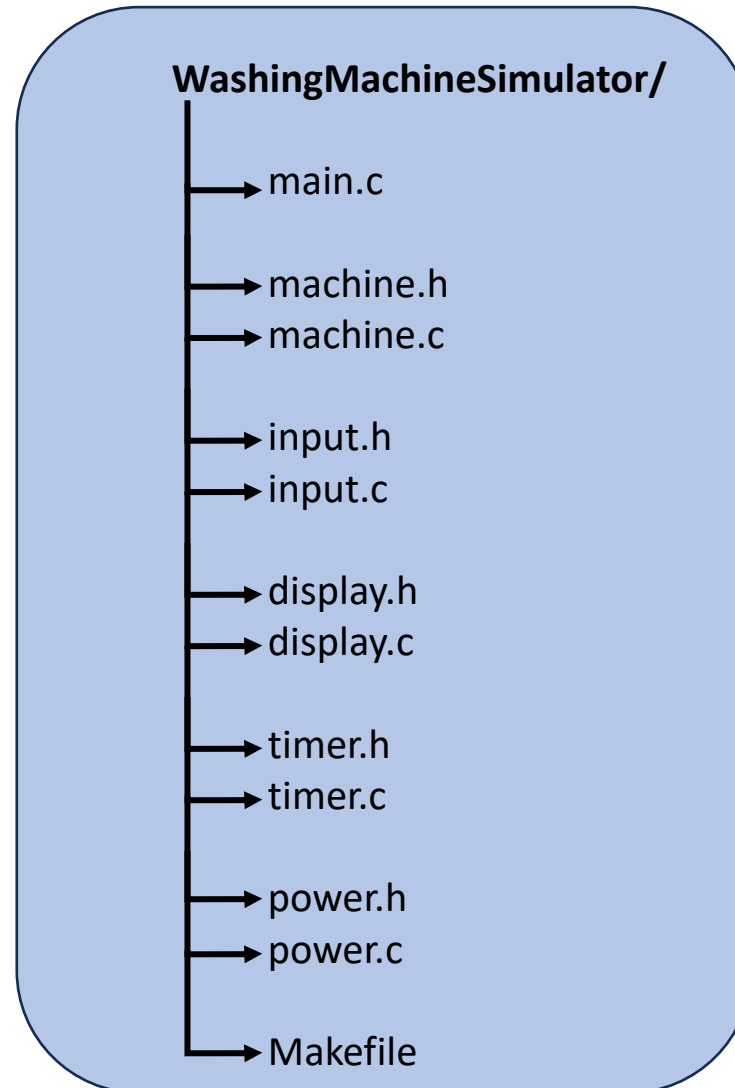


Start Sequence





Power Failure
Sequence



Difference between .h and .c

.h = What a module provides

```
void machine_init(WashingMachine *machine);  
void machine_start(WashingMachine *machine);
```

.c = How the module works

```
void machine_init(WashingMachine *machine)  
{  
    machine->state = IDLE;  
    machine->remaining_time = 0;  
    ...  
}
```


Main controller of the program. It initializes the washing machine, receives user commands, and calls the appropriate module functions based on the input.

```
#include "machine.h"
#include "input.h"
#include "display.h"
#include "timer.h"
#include "power.h"

int main(void)
{
    WashingMachine machine;
    UserInput input;

    machine_init(&machine);

    while (1)
    {
        input = get_user_input();

        switch (input)
        {
            case INPUT_START:
                machine_start(&machine);
                break;

            case INPUT_ABORT:
                machine_abort(&machine);
                break;

            ...

            case INPUT_EXIT:
                return 0;
        }

        if (machine.state == RUNNING)
            timer_tick(&machine);
    }
}
```

Defines the core washing-machine data and interface. It contains the enums, WashingMachine structure, and function declarations required by other modules.

```
#ifndef MACHINE_H
#define MACHINE_H

typedef enum
{
    MODE_NONE,
    MODE_HEAVY,
    MODE_NORMAL,
    MODE_LIGHT
} WashMode;

typedef enum
{
    DOOR_OPEN,
    DOOR_CLOSED,
    DOOR_LOCKED
} DoorStatus;

...
typedef struct
{
    WashMode mode;
    MachineState state;
    DoorStatus door_status;
    int remaining_time;
    int detergent_present;
    int start_requested;
} WashingMachine;

void machine_init(WashingMachine *machine);
void machine_start(WashingMachine *machine);
...

#endif
```

Implements the main washing-machine behaviour.

It handles initialization, mode selection, starting, aborting, door operations, detergent handling, and state transitions.

```
void machine_init(WashingMachine *machine)
{
    machine->mode = MODE_NONE;
    machine->state = IDLE;
    machine->door_status = DOOR_OPEN;
    machine->remaining_time = 0;
    machine->detergent_present = 0;
    machine->start_requested = 0;
}

void machine_start(WashingMachine *machine)
{
    if (machine->state != IDLE)
        return;

    if (machine->mode == MODE_NONE)
        return;

    if (machine->door_status == DOOR_OPEN)
        return;

    if (!machine->detergent_present)
    {
        machine->start_requested = 1;
        machine->state = WAITING_FOR_DETERGENT;
        return;
    }

    ...
}

void machine_select_mode(WashingMachine *machine,
WashMode mode)
{
    if (machine->state == IDLE)
        machine->mode = mode;
}
```

Defines the user-input interface. It contains the UserInput enum and declaration for the function that obtains commands from the user. the commands that can come from the user.

```
#ifndef INPUT_H
#define INPUT_H

typedef enum
{
    INPUT_START,
    INPUT_ABORT,
    INPUT_SELECT_HEAVY,
    INPUT_SELECT_NORMAL,
    INPUT_SELECT_LIGHT,
    INPUT_DOOR_OPEN,
    INPUT_DOOR_CLOSE,
    INPUT_DETERGENT_FILLED,
    INPUT_POWER_OFF,
    INPUT_POWER_ON,
    INPUT_SHOW_STATUS,
    INPUT_EXIT,
    INPUT_INVALID
} UserInput;

UserInput get_user_input(void);

#endif
```

Handles all command-line inputs. It displays the menu, reads the user's selection, validates it, and converts it into a UserInput value.

```
UserInput get_user_input(void)
{
    int choice;

    printf("1. Start\n");
    printf("2. Abort\n");
    printf("3. Select Heavy\n");
    ...

    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            return INPUT_START;

        case 2:
            return INPUT_ABORT;

        case 3:
            return INPUT_SELECT_HEAVY;

        ...

        default:
            return INPUT_INVALID;
    }
}
```

Defines the display interface. It declares functions for displaying the machine's current status and general messages.

```
#ifndef DISPLAY_H
#define DISPLAY_H

#include "machine.h"

void display_status(const WashingMachine *machine);
void display_message(const char *message);

#endif
```

Handles all console output. It reads the machine structure and presents the current mode, state, door status, remaining time, and detergent status to the user.

```
void display_status(const WashingMachine *machine)
{
    printf("Mode      : %s\n", ...);
    printf("State      : %s\n", ...);
    printf("Door       : %s\n", ...);
    printf("Remaining Time : %d\n",
           machine->remaining_time);
    printf("Detergent     : %s\n", ...);
}

const char *state_string(MachineState state)
{
    switch (state)
    {
        case IDLE:
            return "IDLE";

        case RUNNING:
            return "RUNNING";

        ...

        default:
            return "UNKNOWN";
    }
}
```

Defines the timer interface. It declares functions for determining the duration of each wash mode and updating the wash timer.

```
#ifndef TIMER_H
#define TIMER_H

#include "machine.h"

int get_mode_duration(WashMode mode);
void timer_tick(WashingMachine *machine);

#endif
```


Handles wash timing. It assigns the correct duration to Heavy/Normal/Light modes and decrements the remaining time while the machine is running.

```
int get_mode_duration(WashMode mode)
{
    switch (mode)
    {
        case MODE_HEAVY:
            return 45;

        case MODE_NORMAL:
            return 30;

        case MODE_LIGHT:
            return 20;

        ...

        default:
            return 0;
    }
}

void timer_tick(WashingMachine *machine)
{
    if (machine->state != RUNNING)
        return;

    if (machine->remaining_time > 0)
        machine->remaining_time--;

    if (machine->remaining_time == 0)
    {
        ...
        machine->state = COMPLETED;
    }
}
```

Defines the power-management interface. It declares functions for handling power failure and restoration of the washing machine.

```
#ifndef POWER_H
#define POWER_H

#include "machine.h"

void power_failure(WashingMachine *machine);
void power_restore(WashingMachine *machine);

#endif
```

Implements power-failure recovery. It moves the machine into POWER_FAILURE while preserving its current progress and resumes the unfinished cycle when power is restored.

```
void power_failure(WashingMachine *machine)
{
    if (machine->state == RUNNING)
    {
        machine->state = POWER_FAILURE;

        /* remaining_time is preserved */
        machine->door_status = DOOR_LOCKED;
    }
}

void power_restore(WashingMachine *machine)
{
    if (machine->state == POWER_FAILURE)
    {
        /* preserved remaining_time is retained */
        machine->state = RUNNING;
    }
}
```

Automates compilation of the project. It compiles all .c files and links them together to create the final washing-machine executable.

```
CC = gcc
CFLAGS = -Wall -Wextra -std=c11

SRC = main.c machine.c input.c display.c timer.c
      power.c

washing_machine: $(SRC)
                  $(CC) $(CFLAGS) $(SRC) -o washing_machine

clean:
        rm -f washing_machine
```